

Uso de IA:

Prototipo de DNS UI con Stitch

Alcance

Este documento deja constancia de cómo se usó Stitch para crear la primera propuesta visual de la interfaz.

Primero se preparó un prompt breve a partir de los requisitos del proyecto. Luego se refinó y amplió con ChatGPT antes de enviarlo a Stitch.

Prompt original

Diseña una UI web sencilla en español para DNS UI.

Debe permitir crear, editar y eliminar registros DNS de tipo single, multi, weight, round-trip y geo. También debe permitir agregar health checks TCP o HTTP a cada registro.

Incluye otra pantalla llamada IP to Country para administrar registros con IP y país.

Usa un diseño limpio tipo panel administrativo. No agregues login, métricas, gráficas ni funciones extra.

Prompt enviado a Stitch

Diseña un prototipo web de alta fidelidad para una aplicación llamada "DNS UI". Debe ser una interfaz simple, clara y profesional para administrar registros DNS y sus health checks. El idioma completo de la interfaz debe ser español.

Es un prototipo visual de frontend: no incluir conexión con API, base de datos, autenticación, métricas, gráficas ni configuraciones técnicas no solicitadas. Debe ser fácil de implementar después en Next.js/React, sin animaciones llamativas ni componentes innecesarios.

La interfaz debe tener una navegación lateral pequeña con dos secciones:

1. Registros DNS
2. IP to Country

La vista inicial debe ser "Registros DNS".

Vista: Registros DNS

Mostrar:

- Título "Registros DNS".
- Botón principal "Crear registro".
- Una tabla o lista limpia de registros existentes.
- Cada registro debe mostrar: nombre del dominio, tipo de registro y acciones "Editar" y "Eliminar".
- Usar ejemplos visuales simples, sin mostrar datos técnicos que no estén definidos.

- Los cinco tipos de registro deben aparecer exactamente así: **single**, **multi**, **weight**, **round-trip** y **geo**.

Al seleccionar "Crear registro" o "Editar", abrir un panel lateral o modal con:

- Campo "Nombre del dominio".
- Selector visual para elegir el tipo: **single**, **multi**, **weight**, **round-trip** o **geo**.
- La configuración debe cambiar según el tipo seleccionado:
 - **single**: una dirección IP.
 - **multi**: una lista de direcciones IP con acción "Agregar IP".
 - **weight**: una lista de direcciones IP, cada una con su peso asociado.
 - **round-trip**: una lista de direcciones IP que podrán asociarse a health checks.
 - **geo**: una lista de asignaciones de país y dirección IP.

No incluir TTL, puertos, subdominios, proveedores DNS, logs, dashboards, ni campos adicionales no solicitados.

Dentro del mismo panel del registro, incluir una sección clara llamada "Health checks asociados", con un botón "Agregar health check".

Configuración de health checks

Al agregar un health check, mostrar un formulario con selector entre **TCP** y **HTTP**.

Para health check TCP, mostrar únicamente:

- Timeout
- Retries
- Intervalo entre pruebas

Para health check HTTP, mostrar únicamente:

- Path
- Timeout
- Retries
- Intervalo entre pruebas
- Códigos HTTP esperados

Usar etiquetas claras y campos visualmente ordenados. No agregar campos de autenticación básica, puertos, URL completa, headers ni otros datos que no están definidos para el DNS UI.

Vista: IP to Country

Crear una segunda pantalla accesible desde la navegación lateral.

Mostrar:

- Título "IP to Country".
- Botón "Agregar registro".
- Tabla con las columnas "IP", "País" y "Acciones".
- Acciones "Editar" y "Eliminar".

- Modal o panel para crear y editar un registro con los campos “IP” y “País”.

No incluir ciudad, coordenadas, rangos IP, proveedor, continente ni campos adicionales.

Estilo visual

Usar un diseño de panel administrativo sobrio y moderno:

- Fondo claro, tarjetas blancas y bordes suaves.
- Color principal azul oscuro o azul medio.
- Tipografía legible y jerarquía visual clara.
- Botones de acción visibles pero discretos.
- Iconos simples para crear, editar, eliminar y navegar.
- Diseño desktop primero, pero adaptable a pantallas pequeñas.
- Priorizar claridad, espacio en blanco y facilidad de uso sobre decoración.

La interfaz debe verse como una herramienta académica funcional y sencilla, no como un dashboard empresarial con estadísticas.

Aspectos encontrados al implementar el prototipo

El prototipo cumplía el objetivo de definir la apariencia de la aplicación. Sin embargo, al conectarlo con la API aparecieron algunas diferencias entre lo que se había diseñado y la información que el sistema realmente necesita guardar.

Situación	Efecto al implementar	Decisión actual
En el prototipo, IP to Country solo manejaba una IP y un país.	La API trabaja con rangos de IP y también puede guardar el nombre del país, ciudad y coordenadas.	La pantalla se ajustó a esos datos para que la información de la UI no se pierda al guardarla.
El prototipo no incluía TTL.	La API necesita ese valor para crear y editar un registro DNS correctamente.	Se agregó el campo TTL en la UI.
Para round-trip, el prototipo solo pedía direcciones IP.	La API también necesita latitud y longitud para cada dirección.	La UI solicita esos datos solo cuando se selecciona round-trip.
El prototipo permitía agregar health checks asociados.	La versión actual usa una misma configuración de health check para los targets del registro; todavía no permite definir varios checks independientes para una misma IP.	Se mantiene esta forma de trabajo porque es la que soporta la integración actual. Si el requisito final exige varios checks por IP, habrá que ampliarlo.

Conclusión

Stitch se utilizó como apoyo para definir una interfaz inicial clara y sencilla.

Manifiestos de Kubernetes con Claude Code

Alcance

Este apartado deja constancia de cómo se usó Claude (Sonnet 5 y 5.5), mediante Claude Code, para crear los manifiestos YAML de Kubernetes de cada módulo. Se trabajó **un módulo por prompt**, en este orden: `dns-api`, `dns-interceptor`, `health-checker` y `dns-ui`. Cada prompt pidió además aplicar los manifiestos en el cluster local (el Kubernetes de Docker Desktop) y verificarlos, y el resultado quedó en la carpeta `k8s/`.

El segundo, tercer y cuarto prompt reutilizan la frase "mismo estilo y nombres que los anteriores", para que los cuatro módulos queden con la misma estructura (namespace `dns`, etiquetas `app`, imágenes `:local`).

Prompt 1: dns-api

Primero lee el `docker-compose.yml` y los `Dockerfiles` y dime qué cluster local recomiendas y cómo cargar las imágenes. No crees archivos todavía. Después trabajaremos un módulo a la vez, empezando por `dns-api`: genera el Deployment, el Service, el ConfigMap y el Secret con las variables del compose.

Resultado: `k8s/namespace.yaml` y `k8s/dns-api/` con `configmap.yaml`, `secret.example.yaml`, `deployment.yaml` y `service.yaml`. La recomendación fue usar el Kubernetes de Docker Desktop, que comparte las imágenes con Docker (tag `:local` con `imagePullPolicy: IfNotPresent`). Las probes son de tipo TCP porque el API no tiene Actuator. El Secret real se genera desde el `.env` y nunca se versiona; solo se sube la plantilla.

Problemas encontrados:

Problema	Solución
El build de Maven fallaba: un test no compilaba porque <code>RecordResponse</code> cambió de constructor	Se corrigió el test
El pod entraba en <code>CrashLoopBackOff</code> : Hibernate rechazaba la columna <code>int[]</code> de <code>targets</code>	Se cambió el mapeo a <code>@JdbcTypeCode(SqlTypes.ARRAY)</code>
El puerto 8080 estaba ocupado por un contenedor viejo de Compose y el <code>LoadBalancer</code> quedaba en <code><pending></code>	Se detuvo ese contenedor
<code>PUT /api/records</code> daba 500: la columna <code>counter</code> es <code>NOT NULL</code> y el código enviaba <code>null</code>	El contador vale 0 para todos los tipos
Un rolling update dejaba el pod nuevo en <code>CrashLoopBackOff</code> : el pooler de Supabase admite 15 conexiones	Estrategia <code>Recreate</code> y <code>DB_POOL_SIZE</code> configurable

Prompt 2: dns-interceptor

Desarrolla los manifiestos K8s del dns-interceptor, mismo estilo y nombres que los de la dns-api:

- Deployment (imagen local, `imagePullPolicy: IfNotPresent`)

- ConfigMap/Secret con sus variables de entorno (URL de la API = http://:
- Service NodePort, protocolo UDP, port 53, nodePort fijo en 30053
- Probes viables para UDP (sin tcpSocket) Aplícalo, cárgalo al cluster y verifica con nslookup - port=30053 127.0.0.1. No toques docker-compose.yml.

Resultado: `k8s/dns-interceptor/` con `configmap.yaml`, `deployment.yaml` y `service.yaml` (NodePort UDP, 53 → 30053). No se creó un Secret porque su única variable, `DNS_API_URL`, no es sensible. Como UDP no admite `tcpSocket` ni `httpGet`, las probes ejecutan `grep -q ':0035 ' /proc/net/udp`, que comprueba que el socket UDP del puerto 53 siga abierto. Se verificó con `nslookup` contra un registro propio y contra un dominio externo.

Problemas encontrados:

Problema	Solución
En cada redespiegue el pod viejo tardaba unos 35 s en desaparecer y se veían contenedores duplicados: el programa corre como PID 1 sin manejar <code>SIGTERM</code> y Kubernetes espera los 30 s de gracia	Es solo visual; se puede reducir con <code>terminationGracePeriodSeconds</code>
Al activar HTTPS en el API, <code>DNS_API_URL</code> dejó de ser válida	Se cambió a <code>https://dns-api:8443</code>
Un registro con todas sus IPs <code>unhealthy</code> dejaba sin respuesta al cliente	Comportamiento del código del interceptor, señalado al equipo

Prompt 3: health-checker

Desarrolla los manifiestos K8s del health-checker, mismo estilo y nombres que los anteriores:

- Un Deployment por ubicación (mínimo 2), con lat, lon, país y ciudad como variables de entorno distintas por Deployment
- ConfigMap/Secret para la conexión a Supabase, sin valores quemados
- Sin Service externo si no recibe tráfico entrante Aplícalo, cárgalo al cluster y verifica en logs que cada instancia escribe sus pruebas en Supabase. No toques docker-compose.yml.

Resultado: `k8s/health-checker/` con `configmap.yaml` (intervalo de chequeo), `secret.example.yaml` (`DATABASE_URL`), `deployment-cr.yaml` (`CR-01`, Cartago) y `deployment-us.yaml` (`US-01`, Nueva York). No hay Service porque el binario no recibe tráfico. El `DATABASE_URL` reutiliza las credenciales del DNS API, convirtiendo la URL JDBC al formato URI de `libpq`. Se verificó que cada instancia escribe en `health_results` con su propia ubicación: 12 filas de `CR-01` y 12 de `US-01` en el primer ciclo.

Problemas encontrados:

Problema	Solución
<code>kubectll logs</code> de un checker salía vacío: el <code>printf</code> de C queda en buffer cuando escribe a un pipe	El contenedor se ejecuta con <code>stdbuf -oL</code>

Problema	Solución
Los checkers marcaron unhealthy casi todos los registros de ejemplo y el DNS dejó de resolverlos	Se restauraron los flags; los checkers forman parte del sistema y se despliegan encendidos
Se veía el Health Checker sin crear: se había dejado apagado por defecto	El despliegue oficial los enciende (make CHECKERS=off los omite)
El Dockerfile del checker cambió su WORKDIR a /health_checker	Los manifiestos usan rutas relativas y no se vieron afectados

Prompt 4: dns-ui

Desarrolla los manifiestos K8s del dns-ui, mismo estilo y nombres que los anteriores:

- Deployment (imagen local, imagePullPolicy: IfNotPresent)
- ConfigMap con las variables de entorno (conexión a Supabase, sin valores quemados)
- Service NodePort TCP con nodePort fijo en 30080

Aplicalo, cárgalo al cluster y verifica que la UI abre en `http://localhost:30080`. No toques `docker-compose.yml`.

Resultado: `k8s/dns-ui/` con `deployment.yaml` y `service.yaml` (NodePort 30080). **No se creó el ConfigMap que pedía el prompt**, y la IA explicó por qué: la UI no se conecta a Supabase, y su única variable, `NEXT_PUBLIC_DNS_API_URL`, se incrusta al compilar (`--build-arg`), así que un ConfigMap no tendría efecto. Las probes son `httpGet` / porque esa página no consulta la base.

Problemas encontrados:

Problema	Solución
El navegador no podría llamar al API: <code>DNS_UI_ORIGIN</code> (CORS) era <code>http://localhost:3000</code> y la UI se abre en el puerto 30080	Se cambió a <code>http://localhost:30080</code> en el ConfigMap del API
No se pudo comprobar la UI en un navegador real (la extensión de Chrome no estaba conectada)	Se verificó con peticiones HTTP, el origen de la UI y los mismos payloads que arma <code>recordToApi</code>

Aspectos comunes

- Pedir un módulo por prompt y reutilizar el estilo del anterior dio manifiestos consistentes entre sí.
- Pedir que la IA **aplicara y verificara** cada manifiesto hizo aparecer problemas reales que generar el YAML por sí solo no habría mostrado: puertos ocupados, límites de conexiones, buffering de logs y CORS.
- La IA no siguió al pie de la letra dos instrucciones cuando estaban equivocadas (el ConfigMap de la UI y las probes TCP para UDP) y explicó la razón. Conviene revisar siempre esas diferencias.
- Ningún secreto se generó ni se subió al repositorio: los Secrets reales salen de los `.env`, que están ignorados por git, y solo se versionan las plantillas `secret.example.yaml`.

Conclusión

Claude Code se utilizó para generar y probar los manifiestos de los cuatro módulos, y cada resultado se revisó y verificó en el cluster antes de darlo por bueno. Todo el despliegue se automatiza después con el `Makefile` (ver [docu/k8s.md](#)).